TP Module 1 : « Ma collection TCG »

Thème : construire, étape par étape, une application de suivi d'une collection de cartes à collectionner (TCG : *Trading Card Game*).

Stack : Vite + React 19 + JavaScript (JSX)

Fichiers fournis : `fournis/index.css` (la feuille de style) et `fournis/db.json` (les données de l'API).

À la fin du TP, votre application :

- charge une collection de cartes depuis une API et gère le chargement et les erreurs
- affiche les cartes en grille, avec leur rareté, leur prix et leur quantité
- calcule le nombre d'exemplaires et la valeur totale de la collection
- permet de modifier les quantités, de retirer une carte, de rechercher et de filtrer par rareté
- gère des favoris accessibles partout dans l'application
- propose un formulaire d'ajout validé
- offre des raccourcis clavier et un panneau de notes par carte

---

Étape 0 : Mise en place

Consignes

1. Créez le projet et lancez-le :

   ```
   npm create vite@latest tcg-collection -- --template react
   cd tcg-collection
   npm install
   npm run dev
   ```

   Ouvrez l'adresse affichée (en général http://localhost:5173) et laissez ce serveur tourner pendant tout le TP.

2. Initialisez Git et faites un premier commit.
3. Faites le ménage :
   - supprimez `src/App.css` et le dossier `src/assets/`
   - remplacez le contenu de `src/index.css` par celui du fichier fourni `fournis/index.css`
   - dans `index.html`, passez `lang` à `"fr"` et le `<title>` à « Ma collection TCG ».
4. Remplacez `src/App.jsx` par un composant `App` qui affiche, dans un `<main className="app">`, un `<header className="app-header">` contenant un titre `<h1>Ma collection TCG</h1>`.
5. Ouvrez `src/main.jsx` sans le modifier et repérez :
   - `document.getElementById("root")` : l'élément de la page dans lequel React va tout afficher
   - `createRoot(...).render(...)` : React prend le contrôle de la `<div id="root">` d'`index.html`
   - `<StrictMode>` : un mode de développement qui aide à détecter les erreurs.

Vérification

La page affiche « Ma collection TCG » sur fond gris clair, et l'onglet du navigateur porte le même titre. `npm run lint` ne signale rien.

Questions

- **Q0.1** Supprimez temporairement la `<div id="root">` d'`index.html` et rechargez. Quel message apparaît dans la console ? Qu'en déduisez-vous sur le rôle de cette div ? (Remettez-la.)
- **Q0.2** Modifiez le texte du `<h1>` et sauvegardez. La page change sans être rechargée : est-ce React ou Vite qui fait cela ?
- **Q0.3** Combien de fichiers HTML votre application contient-elle ? Que signifie « SPA » ?

Étape 1 : Le domaine

Consignes

1. Créez `src/domain/card.js` et déclarez :
   - Une carte est un objet de la forme `{ id, name, set, rarity, price, quantity }` : `id`, `name` et `set` (l'extension) sont des chaînes, `price` (en euros) et `quantity` (le nombre d'exemplaires possédés) des nombres. Notez cette forme en commentaire en haut du fichier
   - `rarity` vaut toujours l'une des quatre raretés : `"commune"`, `"peu-commune"`, `"rare"` ou `"ultra-rare"`
   - `RARITIES` : le tableau des quatre raretés, figé avec `Object.freeze([...])`
   - `SETS` : `["Set de Base", "Jungle", "Fossile"]`, figé lui aussi
   - `SAMPLE_CARDS` : le tableau ci-dessous, figé avec `Object.freeze`.

id	name	set	rarity	price	quantity
"1"	Dracaufeu	Set de Base	ultra-rare	48.9	1
"2"	Tortank	Set de Base	rare	12.5	2
"3"	Pikachu	Set de Base	commune	0.3	4
"4"	Ronflex	Jungle	ultra-rare	35	1
"5"	Pyroli	Jungle	rare	18.75	1
"6"	Persian	Jungle	peu-commune	1.2	3
"7"	Ptéra	Fossile	rare	9.9	**0**
"8"	Amonistar	Fossile	peu-commune	0.8	2

2. Écrivez et exportez les fonctions pures suivantes :

```
export function parseRarity(value)       // la rareté si value en est une, sinon undefined
export function isRare(card)             // vrai pour "rare" et "ultra-rare"
export function formatPrice(price)       // 48.9 → "48,90 €"
export function copyCount(cards)         // somme des quantités
export function collectionValue(cards)   // somme de prix × quantité
```

   Indices : `RARITIES.find(...)` pour `parseRarity` , `reduce` pour les deux sommes , pour `formatPrice`, créez **une seule fois**, hors de la fonction, un `new Intl.NumberFormat("fr-FR", { style: "currency", currency: "EUR" })`.

3. Dans `App`, sous le titre, affichez un résumé calculé avec ces fonctions, dans un `<p className="summary">` : `8 cartes · 14 exemplaires · 134,05 €`.

Vérification

Le résumé affiche exactement **8 cartes · 14 exemplaires · 134,05 €**.

Questions

- **Q1.1** Dans `App`, essayez `SAMPLE_CARDS.push(SAMPLE_CARDS[0])`. Quelle erreur apparaît dans la console ? Quel est le rôle d'`Object.freeze`, et pourquoi est-ce utile pour des données qu'on ne doit pas modifier ? (Retirez la ligne.)
- **Q1.2** Dans `SAMPLE_CARDS`, remplacez une rareté par `"legendaire"`. Une erreur est-elle signalée ? Que devient la carte à l'écran (couleur, badge) ? Comment pourrait-on détecter ce genre de faute plus tôt ? (Remettez la valeur.)

- **Q1.3** Pourquoi le formateur `Intl.NumberFormat` est-il créé hors de `formatPrice` plutôt qu'à chaque appel ?
- **Q1.4** Dans le JSX, `{SAMPLE_CARDS.length}` affiche 8. Que sont exactement les accolades `{ }` dans du JSX ? Peut-on y écrire un `if` ?

Étape 2 : Premier composant

Consignes

1. Créez `src/components/CardTile.jsx` et exportez un composant `CardTile`, **sans props** pour l'instant, qui affiche toujours la même carte :

```
<article className="card-tile rarity-ultra-rare">
  <h3>Dracaufeu</h3>
  <p className="meta">Set de Base · ultra-rare</p>
  <p className="price">48,90 €</p>
</article>
```

2. Dans `App`, sous le header, affichez **trois** `<CardTile />` dans une `<div className="grid">`.
3. En haut de `App.jsx`, hors du composant, ajoutez : `console.log("Élément :", <CardTile />);`

Vérification

Trois tuiles identiques s'affichent côte à côte. La console affiche un objet.

Questions

- **Q2.1** Dépliez l'objet affiché dans la console. Que contiennent `type` et `props` ? Y a-t-il du HTML dedans ? Comment s'appelle cet objet ?
- **Q2.2** Combien y a-t-il de **composants**, d'**éléments** et d'**instances** de `CardTile` à l'écran ? Définissez chacun des trois termes.
- **Q2.3** Renommez temporairement le composant en `cardTile` (minuscule), à la déclaration et à l'utilisation. Qu'est-ce qui s'affiche, et quel avertissement apparaît dans la console ? Qu'est-ce que cela révèle sur le rôle de la majuscule ?
- **Q2.4** Remplacez temporairement un `<CardTile />` par `{CardTile()}`. L'affichage est identique. Pourquoi est-ce pourtant une mauvaise idée ?

Étape 3 : Props et composition

Consignes

1. **`CardTile`** reçoit maintenant une carte en prop : `function CardTile({ card })`. Affichez les vraies données : le nom, le badge de rareté (point 2), l'extension (`<p className="meta">`) et le prix formaté (`<p className="price">`). La classe de l'`article` devient dynamique : `className={`card-tile rarity-${card.rarity}`}`.
2. Créez **`RarityBadge`** (`src/components/RarityBadge.jsx`) : prop `rarity` (obligatoire) et prop `label` (optionnelle). Il affiche `<span className={`badge badge-${rarity}`}>{label}</span>`. Sans `label`, le libellé est la rareté elle-même : utilisez une **valeur par défaut dans la déstructuration**.
3. Créez **`Panel`** (`src/components/Panel.jsx`) : props `title` et `children`. Il rend une `<section className="panel">` avec un `<h2>` puis ses `children`.
4. Dans `App`, entourez la grille d'un `<Panel title="Mes cartes">` et affichez les trois premières cartes de `SAMPLE_CARDS` (`SAMPLE_CARDS[0]`, `[1]`, `[2]`).
5. Remplacez le `console.log` par `console.log("Élément avec props :", <CardTile card={SAMPLE_CARDS[0]} />);`

Vérification

Trois tuiles différentes (Dracaufeu, Tortank, Pikachu), chacune avec un badge coloré selon sa rareté, dans un panneau blanc titré « Mes cartes ». Dans la console, l'objet a maintenant des `props`.

Questions

- **Q3.1** Retirez la prop `card` d'un `<CardTile />`. Que se passe-t-il, et à quel moment le découvre-t-on : en écrivant le code, au lancement, ou à l'affichage ? Lisez le message d'erreur de la console.
- **Q3.2** Dans `RarityBadge`, comment la valeur par défaut `label = rarity` fonctionne-t-elle ? Passez `label="Légendaire"` sur un badge pour vérifier.
- **Q3.3** Dans `CardTile`, essayez d'écrire `card.name = card.name.toUpperCase(),` avant le `return`. Rien ne vous en empêche. Pourquoi est-ce pourtant interdit en React ? (Retirez la ligne.)
- **Q3.4** Qu'est-ce que `children`, et que permet-il à `Panel` que des props `title` et `body` ne permettraient pas ?

Étape 4 : Listes et rendu conditionnel

Consignes

1. Créez **`CardGrid`** (`src/components/CardGrid.jsx`), prop `cards` (un tableau de cartes) :
   - si la liste est vide, renvoyez directement `<p className="hint">Aucune carte à afficher.</p>` (**early return**) ;
   - sinon, rendez une `<div className="grid">` contenant un `CardTile` par carte, avec `map` et une **`key` stable**.
2. Dans **`CardTile`**, ajoutez deux affichages conditionnels :
   - `★ Carte rare` (`<p className="highlight">`) **seulement** si `isRare(card)`, avec `&&` ;
   - avec un **ternaire** : `× {quantité}` (`<p className="meta">`) si la quantité est supérieure à 0, sinon `Plus aucun exemplaire` (`<p className="sold-out">`).
3. Dans `App`, remplacez les trois tuiles par `<CardGrid cards={SAMPLE_CARDS} />` et supprimez le `console.log`.

Vérification

Les 8 cartes s'affichent. 5 portent la mention « ★ Carte rare ». Ptéra affiche « Plus aucun exemplaire » en rouge. La console ne contient aucun avertissement.

Questions

- **Q4.1** Retirez la `key` du `map`. Quel avertissement apparaît dans la console ? À quoi sert la `key` ? (Remettez-la.)
- **Q4.2** Pourquoi `key={card.id}` et pas `key={index}` (l'index du tableau) ? Dans quel cas l'index serait-il acceptable ?
- **Q4.3** Écrivez temporairement `{card.quantity && <p className="meta">× {card.quantity}</p>}`. Qu'affiche la tuile de Ptéra ? Pourquoi ? Comment s'en protéger ?
- **Q4.4** Passez temporairement `cards={[]}` à `CardGrid`. Que voit-on ?
- **Q4.5** Sans modifier `SAMPLE_CARDS`, affichez uniquement les cartes rares. Quelle méthode de tableau utilisez-vous, et pourquoi ne modifie-t-elle pas le tableau d'origine ?

Étape 5 : Le state

Consignes

1. Créez **`QuantityControl`** (`src/components/QuantityControl.jsx`), prop `initialQuantity` (un nombre) :
   - déclarez un state `quantity` initialisé avec `initialQuantity`
   - affichez un bouton `−`, la quantité dans un `<strong>`, puis un bouton `+`, le tout dans une `<div className="quantity-control">`
   - écrivez deux handlers, `handleDecrement` et `handleIncrement`
   - le bouton `−` est désactivé quand la quantité vaut 0
   - donnez aux boutons des `aria-label` (« Retirer un exemplaire », « Ajouter un exemplaire ») et `type="button"`.
2. Dans **`CardTile`**, remplacez le ternaire de l'étape 4 :
   - gardez seulement `Plus aucun exemplaire` quand `card.quantity === 0` (avec `&&`)
   - ajoutez `<QuantityControl initialQuantity={card.quantity} />` en bas de la tuile.

Vérification

Chaque tuile a son compteur. Cliquer sur `+` d'une carte ne change que cette carte. Mais observez bien : **le résumé du header ne bouge pas**, et la mention « Plus aucun exemplaire » de Ptéra reste affichée même après un clic sur `+`. C'est normal : l'étape 6 corrige cela.

Questions

- **Q5.1** Remplacez temporairement le state par une variable : `let quantity = initialQuantity;` et `quantity = quantity + 1` dans le handler. Que se passe-t-il au clic ? Donnez **deux** raisons.
- **Q5.2** Ajoutez `console.log(quantity)` juste **après** `setQuantity(quantity + 1)` dans `handleIncrement`. Quelle valeur s'affiche : l'ancienne ou la nouvelle ? Pourquoi ?
- **Q5.3** Dans `handleIncrement`, appelez **deux fois** `setQuantity(quantity + 1)`. De combien la quantité augmente-t-elle ? Corrigez avec la forme fonctionnelle `setQuantity((q) => q + 1)` et expliquez la différence.
- **Q5.4** Placez temporairement le `useState` dans un `if (initialQuantity >= 0) { … }`. Que disent ESLint et React ? Pourquoi cette règle existe-t-elle ?
- **Q5.5** Pourquoi le résumé du header ne peut-il pas connaître les quantités affichées par les compteurs ?

Étape 6 : Remonter le state et l'immutabilité

La collection doit maintenant vivre **à un seul endroit**, lu par la grille **et** par le résumé : leur plus proche ancêtre commun, `App`.

Consignes

1. Dans **`App`** :
   - déclarez `const [cards, setCards] = useState(SAMPLE_CARDS);`
   - écrivez `handleQuantityChange(id, delta)` : avec `map` et le spread d'objet, la carte visée reçoit `quantity: Math.max(0, card.quantity + delta)`, les autres restent inchangées
   - écrivez `handleRemove(id)` avec `filter`
   - utilisez la **forme fonctionnelle** du setter (`setCards((previous) => …)`).
2. **`QuantityControl`** ne possède plus rien : ses props deviennent `quantity` (un nombre) et `onChange` (une fonction appelée avec `+1` ou `-1`). Les boutons appellent `onChange(-1)` et `onChange(1)`. Supprimez le `useState`.
3. **`CardTile`** reçoit une prop optionnelle `children`, affichée en bas dans une `<div className="actions">`, seulement s'il y en a.
4. **`CardGrid`** reçoit `onQuantityChange` et `onRemove`, et passe à chaque `CardTile`, en `children`, un `QuantityControl` et un bouton « Retirer ».
5. Créez **`Summary`** (`src/components/Summary.jsx`), prop `cards`, qui affiche dans un `<p className="summary">` le nombre de cartes, le nombre d'exemplaires et la valeur, **sans aucun state**. Utilisez-le dans le header à la place du résumé de l'étape 1.

Vérification

- Cliquer sur `+` ou `−` met à jour la tuile **et** le header (nombre d'exemplaires et valeur).
- Ptéra perd sa mention « Plus aucun exemplaire » dès qu'il a un exemplaire.
- « Retirer » fait disparaître la carte et met le header à jour ; quand tout est retiré, « Aucune carte à afficher. » apparaît.

Questions

- **Q6.1** Remplacez temporairement le corps de `handleQuantityChange` par une **mutation** :

  ```
  setCards((previous) => {
    const card = previous.find((c) => c.id === id);
    if (card) card.quantity += delta;
    return previous;
  });
  ```

  Que se passe-t-il à l'écran ? Pourquoi ? `Object.freeze` protège `SAMPLE_CARDS` : pourquoi n'a-t-il pas empêché cette mutation ? (Remettez le `map`.)
- **Q6.2** Pourquoi `Summary` n'a-t-il besoin d'aucun state ? Que risquerait-on en stockant la valeur totale dans un `useState` ?
- **Q6.3** Dans `CardGrid`, pourquoi écrit-on `onChange={(delta) => onQuantityChange(card.id, delta)}` plutôt que de passer `onQuantityChange` directement à `QuantityControl` ?
- **Q6.4** Ajoutez dans le header un bouton « Tout à 1 exemplaire » qui remet toutes les quantités à 1, avec un seul `map`.
- **Q6.5** Faites le schéma de l'arbre des composants. Indiquez par une flèche descendante les données qui descendent et par une flèche montante les intentions qui remontent.

Étape 7 : La recherche (lifting state up)

Consignes

1. Créez **`SearchBar`** (`src/components/SearchBar.jsx`), **sans aucun state**. Ses props :

   ```
   // Props de SearchBar :
   //   query           la recherche en cours (un texte)
   //   onQueryChange   appelée avec le nouveau texte
   //   rarity          la rareté choisie, ou "toutes"
   //   onRarityChange  appelée avec la nouvelle rareté
   export function SearchBar({ query, onQueryChange, rarity, onRarityChange }) { … }
   ```

   Elle rend une `<div className="toolbar" role="search">` avec :

   - un `<label>Recherche <input type="search" … /></label>` contrôlé (`value` + `onChange`)
   - un `<label>Rareté <select>…</select></label>` contrôlé, avec une option « Toutes » (`value="toutes"`) puis une option par rareté (`RARITIES.map`). Convertissez `event.target.value` avec `parseRarity(...) ?? "toutes"`.

2. Dans **`App`** :
   - ajoutez deux states, `query` (`""`) et `rarity` (`"toutes"`)
   - calculez `visibleCards` : les cartes dont le nom contient la recherche (sans tenir compte de la casse ni des espaces autour) **et** qui correspondent à la rareté choisie
   - placez la `SearchBar` en haut du panneau, passez `visibleCards` à la grille, et affichez dans le titre du panneau `Mes cartes (visibles / total)`.

Vérification

- Taper « dracaufeu » n'affiche que Dracaufeu, et le titre devient « Mes cartes (1 / 8) ».
- Choisir « ultra-rare » affiche 2 cartes. Les deux filtres se combinent.
- Le résumé du header porte toujours sur **toute** la collection.

Questions

- **Q7.1** Pourquoi le state `query` ne peut-il pas vivre dans `SearchBar` ? Qu'est-ce qui casserait ?
- **Q7.2** Pourquoi `visibleCards` n'est-il pas un `useState` ? Décrivez le bug que l'on aurait en le stockant.
- **Q7.3** On passe `onQueryChange={setQuery}` directement, sans écrire de fonction fléchée. Pourquoi est-ce possible ?
- **Q7.4** Étendez la recherche à l'extension : « fossile » doit afficher les cartes de Fossile.
- **Q7.5** Filtrez sur « rare », puis retirez une carte rare. Le titre du panneau est-il juste ? Pourquoi n'a-t-on rien eu à coder pour cela ?

Étape 8 : Les effets

Consignes

1. Dans **`App`**, calculez `const value = collectionValue(cards);` et **synchronisez le titre de l'onglet** avec : `document.title = \`Ma collection (${formatPrice(value)})\``, dans un `useEffect` qui dépend de `value`.
2. Dans **`SearchBar`**, ajoutez un **raccourci clavier** : la touche Échap vide la recherche, où que soit le focus.
   - Dans un `useEffect`, abonnez-vous à `window.addEventListener("keydown", handleKeyDown)`.
   - Renvoyez une fonction de **cleanup** qui retire l'écouteur.
   - Ajoutez un `console.log` dans le setup (« écouteur installé ») et dans le cleanup (« écouteur retiré »).
   - Déclarez honnêtement les dépendances : la fonction de l'effet lit `onQueryChange`.

Vérification

- Le titre de l'onglet affiche la valeur et suit les changements de quantité.
- Taper une recherche puis appuyer sur Échap vide le champ.
- Au chargement, la console affiche « installé → retiré → installé ».

Questions

- **Q8.1** Expliquez la séquence « installé → retiré → installé » au chargement. Se produira-t-elle en production ?
- **Q8.2** Supprimez temporairement le `return` du cleanup et ajoutez un `console.log("Échap")` dans `handleKeyDown`. Rechargez, puis appuyez sur Échap. Combien de fois le message s'affiche-t-il ? Quel problème le cleanup évite-t-il ?
- **Q8.3** Remplacez temporairement `[value]` par `[]`. Que fait le titre quand la valeur change ? Que dit ESLint ? Que se passe-t-il sans aucun tableau ?
- **Q8.4** Un camarade propose de stocker `value` dans un `useState` mis à jour par un `useEffect` qui dépend de `cards`. Donnez deux raisons de refuser.
- **Q8.5** Pourquoi la modification du titre ne peut-elle pas être écrite directement dans le corps du composant, hors de tout effet ?

Étape 9 : Charger depuis une API

Consignes

1. Installez json-server, copiez `fournis/db.json` à la **racine** du projet (12 cartes) et ajoutez le script dans `package.json` :

   ```
   npm install -D json-server

   "api": "json-server db.json --port 3001"
   ```

   Dans un **second terminal**, lancez `npm run api` et ouvrez http://localhost:3001/cards : vous devez voir le JSON.

2. Créez **`src/api/cards.js`** avec `fetchCards()`, une fonction `async` qui fait un `GET http://localhost:3001/cards` et renvoie la liste des cartes.
   - **`fetch` ne rejette pas sur un 404 ou un 500** : si `!response.ok`, levez `new Error(`HTTP ${response.status}`)`.
   - Renvoyez `response.json()`. Rien ne vérifie la forme des données reçues : notez-le en commentaire.
3. Dans **`App`**, remplacez le state `cards` par un state qui modélise la requête :

   ```
   // L'état de la requête prend toujours l'une de ces trois formes :
   { status: "loading" }
   { status: "error", message: "…" }
   { status: "success", cards: [ … ] }
   ```

   - Dans un `useEffect`, appelez `fetchCards()` et passez en `success` ou en `error`. Protégez-vous avec un **drapeau `ignore`**, mis à `true` dans le cleanup.
   - En cas d'erreur, JavaScript peut lancer n'importe quelle valeur, pas seulement une `Error` : utilisez `error instanceof Error ? error.message : String(error)`.
   - Ajoutez un bouton « Réessayer » : un state `attempt`, ajouté aux dépendances de l'effet, que le bouton incrémente. C'est **le handler** qui repasse le state en `loading`.
   - Écrivez une fonction `updateCards(transform)` qui ne transforme les cartes que si le state est en `success`, et réécrivez `handleQuantityChange` et `handleRemove` avec elle.
   - **Tous les hooks restent avant les `return` anticipés** : dérivez `const cards = state.status === "success" ? state.cards : [];` avant l'effet du titre.
   - Affichez « Chargement de la collection… » pendant le chargement, et le message d'erreur avec le bouton « Réessayer » en cas d'échec.
4. `SAMPLE_CARDS` n'est plus utilisé dans `App`. Gardez-le dans le domaine : il resservira pour les tests au module suivant.

Vérification

- Les **12** cartes de l'API s'affichent. Le header indique **12 cartes · 26 exemplaires · 202,50 €**.
- Dans l'onglet Réseau (F12), activez le throttling « 3G lent » et rechargez : « Chargement… » est visible.
- Arrêtez l'API (Ctrl+C dans son terminal) et rechargez : le message d'erreur s'affiche. Relancez l'API et cliquez sur « Réessayer » : la collection apparaît.

Questions

- **Q9.1** Pourquoi ne peut-on pas écrire `useEffect(async () => { … })` ?
- **Q9.2** Remplacez temporairement l'URL par `/cardz`. Quel message s'affiche ? Sans le test `!response.ok`, que se passerait-il ?
- **Q9.3** Dans l'onglet Réseau, combien de requêtes `cards` partent au chargement en développement ? Pourquoi n'est-ce pas un bug, et quel rôle joue `ignore` ici ?

- **Q9.4** Décrivez un scénario de **race condition** que le drapeau `ignore` empêche. Aide : imaginez que l'effet dépende d'un identifiant qui change pendant qu'une requête est en vol.
- **Q9.5** Déplacez temporairement le `useEffect` du titre **après** le `if (state.status === "loading") return …`. Que dit ESLint ? Pourquoi ?
- **Q9.6** Pourquoi un seul state avec un champ `status` plutôt que trois states `isLoading`, `error` et `cards` ? Citez un état absurde que cette forme rend impossible.
- **Bonus (sans corrigé)** : faites en sorte que la modification d'une quantité soit aussi envoyée au serveur (`PATCH /cards/:id` avec `{ quantity }`), dans `src/api/cards.js`.

Étape 10 : Les hooks personnalisés

Consignes

1. Créez **`src/hooks/useCards.js`** et déplacez-y tout ce qui concerne la collection : les states `state` et `attempt`, l'effet de chargement, `updateCards`, et les actions renommées `changeQuantity`, `removeCard` et `retry`. Le hook renvoie `{ state, retry, changeQuantity, removeCard }`.
2. Créez **`src/hooks/useDocumentTitle.js`** : `useDocumentTitle(title)` synchronise `document.title` et, dans son cleanup, **restaure le titre précédent**.
3. Simplifiez **`App`** : `const { state, retry, changeQuantity, removeCard } = useCards();` et `useDocumentTitle(...)`.

Vérification

L'application se comporte exactement comme à l'étape 9. `App.jsx` ne contient plus de `useEffect`, et `npm run lint` passe.

Questions

- **Q10.1** Pourquoi ces fichiers ont-ils l'extension `.js` et non `.jsx` ?
- **Q10.2** Appelez temporairement `useCards()` **deux fois** dans `App` et affichez deux grilles. Combien de requêtes partent ? Modifiez une quantité dans la première grille : la seconde change-t-elle ? Qu'en concluez-vous sur ce qu'un hook partage ?
- **Q10.3** Que se passerait-il si le hook s'appelait `loadCards` au lieu de `useCards` ? (Pensez au linter.)
- **Q10.4** `useDocumentTitle` ne renvoie rien. À quoi sert-il alors ?

Étape 11 : Les favoris en Context

Consignes

1. Créez **`src/context/favorites.js`** :
   - la valeur du contexte sera un objet `{ favoriteIds, toggleFavorite }` : la liste des ids favoris, et la fonction qui ajoute ou retire un favori
   - `FavoritesContext = createContext(null)` : `null` représente « en dehors de tout provider »
   - le hook `useFavorites()`, qui lit le contexte et **lève une erreur explicite** s'il vaut `null` (« useFavorites doit être appelé sous <FavoritesProvider> »).
2. Créez **`src/context/FavoritesProvider.jsx`** : un composant qui **possède** le state `favoriteIds` et la fonction `toggleFavorite` (ajoute l'id s'il est absent, le retire sinon, sans mutation), et qui rend `<FavoritesContext value={{ favoriteIds, toggleFavorite }}>{children}</FavoritesContext>`. En React 19, le contexte s'utilise directement comme provider, sans `.Provider`.
3. Dans **`main.jsx`**, entourez `<App />` de `<FavoritesProvider>`.
4. Créez **`FavoriteButton`** (prop `cardId`) : il affiche ★ ou ☆ selon l'état, bascule au clic, avec `aria-pressed` et un `aria-label` (« Ajouter aux favoris » ou « Retirer des favoris »). Placez-le **dans `CardTile`**, à côté du nom (regroupez le `<h3>` et le bouton dans une `<div className="actions">`).
5. Créez **`FavoritesCount`**, qui affiche `★ N favori(s)`, et placez-le dans le header.

Vérification

- Cliquer sur ☆ d'une carte la passe en ★ et incrémente le compteur du header.
- Aucune prop liée aux favoris n'apparaît dans `App`, `CardGrid` ni `CardTile` (hormis le `cardId` passé au bouton).

Questions

- **Q11.1** Sans Context, quelles props auriez-vous dû ajouter, et à quels composants ?
- **Q11.2** Dans `main.jsx`, rendez temporairement `<FavoritesCount />` **hors** du provider. Que se passe-t-il ? Pourquoi est-ce mieux qu'un `undefined` silencieux ?
- **Q11.3** Le Context « possède »-t-il les favoris ? Où vivent-ils réellement ?
- **Q11.4** Quand un favori change, quels composants se rendent de nouveau ? Pourquoi réserve-t-on le Context aux données qui changent peu ?
- **Bonus** : conservez les favoris dans le `localStorage` pour qu'ils survivent au rechargement (un effet de synchronisation, et un initialiseur paresseux `useState(() => …)` pour la lecture). L'app de référence du cours en donne un exemple.

Étape 12 : Le formulaire d'ajout

Consignes

1. Dans **`useCards`**, ajoutez et exportez `addCard(newCard)` : la carte (sans id) est ajoutée en fin de liste avec un id `crypto.randomUUID()` (le serveur le fera au module suivant).
2. Créez **`AddCardForm`** (`src/components/AddCardForm.jsx`), prop `onAdd` : une fonction appelée avec la nouvelle carte.
   - **Le brouillon** : un seul objet de state, de la forme

     ```
     { name: "", set: "Set de Base", rarity: "commune", price: "", quantity: "1" }
     ```

     Le prix et la quantité restent des **chaînes** pendant la saisie, et sont convertis à la validation. Prévoyez une constante `EMPTY_DRAFT` (extension `SETS[0]`, rareté `"commune"`, prix vide, quantité `"1"`).
   - **Un handler unique** :

     ```
     function update(field, value) { … }   // field : "name", "set", "rarity", "price" ou "quantity"
     ```
   - **La validation** : une fonction pure `validate(draft)`, hors du composant, qui renvoie la liste des messages d'erreur, avec trois règles : nom d'au moins 2 caractères (sans les espaces autour) ; prix numérique positif ou nul ; quantité entière supérieure ou égale à 1. Les erreurs sont **calculées à chaque rendu**, jamais stockées.
   - Un state `submitted` : les erreurs ne s'affichent qu'**après** une première tentative d'envoi.
   - **`handleSubmit`** : `event.preventDefault()`, puis `submitted` passe à `true` ; s'il y a des erreurs, on s'arrête. Sinon on appelle `onAdd` avec la carte convertie (prix et quantité en nombres), on remet le brouillon à `EMPTY_DRAFT` et `submitted` à `false`.
   - **Les champs**, tous contrôlés, dans un `<form className="add-form" noValidate>` : Nom (`input`), Extension (`select` sur `SETS`), Rareté (`select` sur `RARITIES`, avec `parseRarity`), Prix (`input` avec `inputMode="decimal"`), Quantité (`input type="number"`), puis un bouton « Ajouter la carte » en `type="submit"`. Affichez les erreurs dans une `<ul className="errors error">`.
3. Dans **`App`**, ajoutez un `<Panel title="Ajouter une carte">` contenant le formulaire, au-dessus de la collection.

Vérification

- Cliquer sur « Ajouter la carte » avec un formulaire vide affiche les erreurs (nom, prix).
- Avec « Évoli », prix « 3.5 » : la carte apparaît dans la grille, le compteur passe à 13, et le formulaire est vidé.
- Appuyer sur Entrée dans un champ soumet aussi le formulaire.

Questions

- **Q12.1** Retirez temporairement `event.preventDefault()`, ajoutez une carte et regardez l'URL. Que se passe-t-il, et qu'advient-il de la carte ajoutée ?
- **Q12.2** Écrivez temporairement `update("nme", "x")` (avec une faute de frappe). Que se passe-t-il ? Inspectez le state avec React DevTools : que contient maintenant le brouillon ? Pourquoi aucune erreur ne le signale ?
- **Q12.3** Pourquoi les erreurs ne sont-elles pas dans un `useState` ?
- **Q12.4** Ajoutez un bouton « Vider » qui remet le brouillon à zéro. Quel `type` doit-il avoir, et pourquoi ?
- **Q12.5** Chaque frappe provoque un nouveau rendu de tout le formulaire. Est-ce un problème ?
- **Q12.6** Pourquoi `parseRarity(...) ?? "commune"` plutôt que d'utiliser directement `event.target.value` ?

Étape 13 : Les refs

Consignes

1. Dans **`SearchBar`** :
   - créez `const inputRef = useRef(null);` et branchez-la sur le champ de recherche avec `ref={inputRef}` ;
   - ajoutez un bouton « Effacer » (`type="button"`, désactivé si la recherche est vide) qui vide la recherche **et redonne le focus au champ** (`inputRef.current?.focus()`) ;
   - dans l'effet clavier de l'étape 8, ajoutez le raccourci **« / »** : s'il est pressé alors que le focus n'est pas déjà dans le champ, il appelle `event.preventDefault()` et donne le focus au champ. Retirez les `console.log` de l'étape 8.
2. Dans **`AddCardForm`**, après un ajout réussi, **remettez le curseur dans le champ Nom** pour enchaîner les saisies au clavier.

Vérification

- « / » place le curseur dans la recherche, sans écrire de « / ».
- « Effacer » vide la recherche et le curseur reste dans le champ.
- Après un ajout, le curseur est dans le champ Nom.

Questions

- **Q13.1** Ajoutez `console.log(inputRef.current)` dans le corps de `SearchBar`, juste avant le `return`. Que vaut-il au premier rendu ? Et dans `handleClear` ? Pourquoi ?
- **Q13.2** Pourquoi une ref et pas un state pour garder le nœud DOM ?
- **Q13.3** Créez temporairement un compteur de rendus dans `App` : `const renders = useRef(0); renders.current++;` puis affichez `renders.current`. Pourquoi est-ce impossible avec un state ? Pourquoi le compteur avance-t-il de 2 en 2 ? (Retirez-le ensuite : modifier une ref pendant le rendu est normalement interdit.)
- **Q13.4** Une ref, une variable locale et un state : pour chacun, dites s'il survit aux rendus et s'il en déclenche un.

Étape 14 : Pour aller plus loin : le reset par key

On ajoute un panneau de notes : on choisit une carte, on tape une note libre (non enregistrée).

Consignes

1. Créez **`CardNotes`** (prop `card`) : un `Panel` titré `Notes : {nom}` qui contient une `<div className="details">` avec une `textarea` contrôlée (state local `note`) et `<p className="hint">N caractère(s), non enregistré.</p>`.
2. Dans **`App`** :
   - ajoutez un state `selectedId`, qui vaut `null` tant qu'aucune carte n'est sélectionnée ;
   - **dérivez** `selectedCard = cards.find((card) => card.id === selectedId)`, sans stocker l'objet carte ;
   - affichez `{selectedCard && <CardNotes key={selectedCard.id} card={selectedCard} />}` entre le formulaire et la collection.
3. **`CardGrid`** reçoit `selectedId` et `onSelect(id)`, et ajoute un bouton « Notes » dans les actions de chaque tuile. **`CardTile`** reçoit une prop optionnelle `selected` (`false` par défaut) qui ajoute la classe `is-selected`.

Vérification

- « Notes » sur une carte ouvre son panneau et encadre sa tuile.
- Tapez une note sur Tortank, puis ouvrez les notes de Ptéra : le panneau de Ptéra est **vide**.
- Retirez la carte sélectionnée : le panneau disparaît de lui-même.

Questions

- **Q14.1** Retirez temporairement `key={selectedCard.id}`. Refaites la vérification « Tortank puis Ptéra » : que devient la note ? Expliquez avec la notion d'instance (type et position dans l'arbre).
- **Q14.2** Pourquoi stocker `selectedId` plutôt que l'objet `selectedCard` lui-même ? Que se passerait-il après un changement de quantité de la carte sélectionnée ?
- **Q14.3** Pourquoi le panneau disparaît-il quand on retire la carte sélectionnée, alors qu'aucun code ne remet `selectedId` à `null` ?
